C200 Programming Assignment № 4

Dr. M.M. Dalkilic
Computer Science
School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

February 10, 2024

Introduction

Due Date: 11:00 PM, Friday, February 16th 2024 Submit your work to the Autograder `https://c200.luddy.indiana.edu/` and remember to add, commit and **push often** to GitHub **before the deadline. We do not accept late submissions.**

In this homework, you'll start mastering your skill in writing functions. Make sure to start working early on this assignment. If we do not explictly mentioned a function/library to use, you are not allowed to use it. For example, if we have not mentioned to use tkinter library then you should not import tkinter in your python file. If you're found cheating...well, you know the consequences by now which includes using tools to solve *any* portion of any problem.

Instructions

1. Make sure that you are **following the instructions** in the PDF, especially the format of output returned by the functions. For example, if a function is expected to return a numerical value, then make sure that a numerical value is returned (not a list or a dictionary). Similarly, if a list is expected to be returned then return a list (not a tuple, set or dictionary).

2. Test **debug** the code well (syntax, logical and implementation errors), before submitting to the Autograder. These errors can be easily fixed by running the code in VSC and watching for unexpected behavior such as, program failing with syntax error or not returning correct output.

3. Make sure that the **code does not have infinite loop (that never exits)** before submitting to the Autograder. You can easily check for this by running in VSC and watching for program output, if it terminates timely or not.

4. Given that you already tried points 1-3, if you see that Autograder does not do anything (after you press 'submit') and waited for a while (30 seconds to 50 seconds), try refreshing the page or using a different browser.

5. Once you are done testing your code, comment out the tests i.e. the code under the __name__ == "__main__" section.

Problem 1: Day of the Week

The modulus operator is denoted by the symbol %. For $x\%y$ it returns the remainder after y is divided into x a whole number of times. While you've seen this before, the format is likely different. To refresh your memory you can visit `https://realpython.com/python-modulo-operator/` or do your own search. For this problem, you will also need to use the floor function in python - think of floor as a way to round down a floating point number to the nearest integer. For example, floor(18.90) is 18 and floor(14.25) is 14.

An approximate (works generally okay for a number of dates, but **not** all) formula for computing the day of the week given dlst = [d,m,y] where d is day, m is month, and y is year is:

$$dlst = [d, m, y] \tag{1}$$

$$a(dlst) = y - \frac{(14-m)}{12} \tag{2}$$

$$x = a(dlst) + \frac{a(dlst)}{4} - \frac{a(dlst)}{100} + \frac{a(dlst)}{400} \tag{3}$$

$$b(dlst) = floor(x) \tag{4}$$

$$c(dlst) = m + 12(\frac{14-m}{12}) - 2 \tag{5}$$

$$day((d, m, y)) = (d + b(dlst) + (31 \cdot \frac{c(dlst)}{12}))\%7 \tag{6}$$

The function day returns a number $1, 2, \ldots, 7$ where $1 = Monday, 2 = Tuesday, \ldots$. If we use this number as the key with the week dictionary, we are able to return the correct day of the week. Here is the dictionary used:

```
week = {1:"Mon", 2:"Tue", 3:"Wed", 4:"Thu", 5:"Fri", 6:"Sat", 7:"Sun"}
```

For example,

$$day([14, 2, 2000]) = \text{Mon} \tag{7}$$

$$print(day([14, 2, 1963])) = \text{Thu} \tag{8}$$

$$print(day([14, 2, 1972])) = \text{Mon} \tag{9}$$

2/14/2000 falls on a Monday; 2/14/1963 falls on a Thursday; 2/14/1972 falls on a Monday.

Deliverables for Problem 1

- Complete the functions described above.
- For calculating b(dlst), you can use the math.floor() function from the math library.
- Remember, the day function utilizes the week dictionary.

Problem 2: Starting quantum computing and quantitative finance

Quantum Computing is a different model of computation imagined by the physicist Feynman. Instead of whole numbers, like we use in the Turing model, it uses complex numbers. Complex numbers, if you remember, are actually pairs of real numbers written as:

$$\text{complex number} \quad = \quad x \pm y\, i \tag{10}$$

where $x, y \in \mathbb{R}$ (they're just numbers) and there's a lone $i = \sqrt{-1}$. The x is called the real part and the y is called the imaginary part. For example,

$$x^2 + 1 \quad = \quad 0 \tag{11}$$

$$x \quad = \quad \sqrt{-1} = i \tag{12}$$

Then

$$x^2 + 1 \quad = \quad (\sqrt{-1})^2 + 1 = -1 + 1 = 0 \tag{13}$$

In this problem: we will learn to return the complex roots for an equation. In Python we use the function complex(x,y) to form a complex number $(x \pm yj)$ where j represents i and there's no space between the $\pm$ sign and numbers. Let's write the function for the values shown on (10), build the complex number, and show it's solution.

```
>>> def f(x):
...     return x**2 + 1
...
>>> root = complex(0,1)
>>> f(root)
0j
>>> f(root) == 0
True
>>> f(root).real
0.0
>>> f(root).imag
0.0
```

Let's try to look at an example: for a quadratic from last homework, you know that the answer is either real or complex. To remind you, the answer is complex when the discriminant is negative. Fig. 1. provide a visualization of two curves (blue curve has imaginary roots, red has real roots)–the blue curve for complex roots does **intersect** the abscissa. Observe that you'll get zero (on the x-axis or *abscissa*) if you put either of the highlighted x values in the equation for red curve. Sometimes the discriminant (the value in the squareroot) is negative. This is where i comes in. We simply multiply the value by -1, thereby allowing us to take the squareroot, but then we append an i to signal it's imaginary.

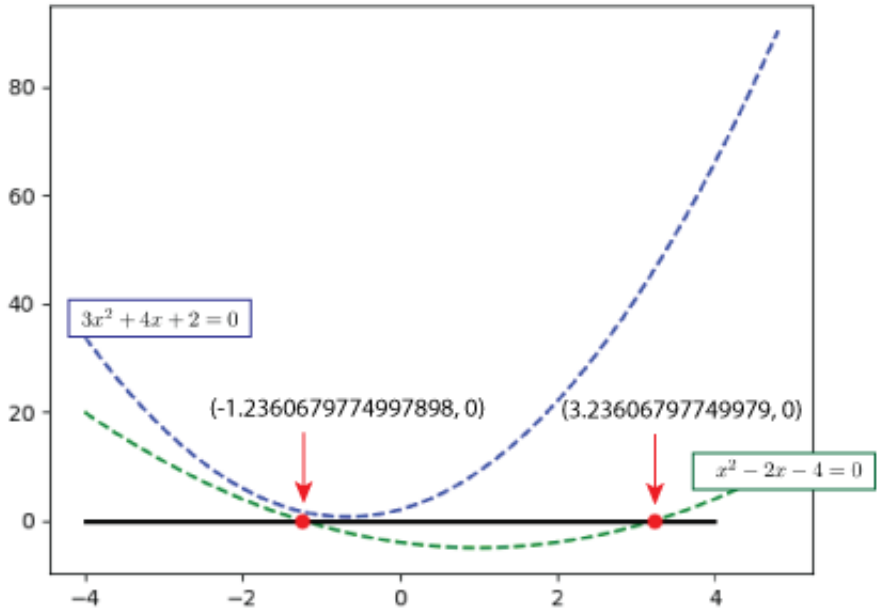


Figure 1: The red dots (with red arrows) are where your solutions are to $x^2 - 2x - 4 = 0$. The dash curve is the function itself. The horizontal line just makes it easier to see the x-axis. The two values are -1.2360679774997898 and 3.23606797749979. For the other function $3x^2 + 4x + 2 = 0$ shown in blue, because it has an imaginary part, it cannot cross the axis.

For example, suppose we have $3x^2 + 4x + 2 = 0$. Then we find, after some algebra:

$$
\begin{aligned}
x &= \frac{-4 \pm \sqrt{-8}}{6} && (14)\\
&= \frac{-4 \pm \sqrt{-2 \times 4}}{6} && (15)\\
&= \frac{-4 \pm \sqrt{-2} \times \sqrt{4}}{6} && (16)\\
&= \frac{-4 \pm 2\sqrt{-2}}{6} && (17)\\
&= -\tfrac{2}{3} \pm \tfrac{\sqrt{2}}{3} i && (18)\\
&= (-\tfrac{2}{3} + \tfrac{\sqrt{2}}{3} i, -\tfrac{2}{3} - \tfrac{\sqrt{2}}{3} i) && (19)
\end{aligned}
$$

In Python we would write:

```
>>> import math
>>> complex(-2/3,math.sqrt(2)/3),complex(-2/3,-math.sqrt(2)/3)
((-0.6666666666666666+0.47140452079103173j), ↩
    (-0.6666666666666666-0.47140452079103173j))
>>> x = round(-2/3,2)
>>> y = round(math.sqrt(2)/3)
>>> complex(x,y), complex(x,-y)
((-0.67+0j), (-0.67+0j))
>>> def f(x):
...     return 3*(x**2) + 4*x + 2
...
```

```
11 >>> f(complex(x,y))
12 (0.6667000000000001+0j)
13 >>> f(complex(x,-y))
14 (0.6667000000000001+0j)
15 >>> f(complex(-2/3,math.sqrt(2)/3))
16 0j
```

Observe the difference between using the full root and only to two decimal places. You'll write a function q(t) where:

$$q((a,b,c)) = \begin{cases} (r_0, r_1) & \text{real roots if } b^2 - 4ac \geq 0 \\ (c_0, c_1) & \text{complex roots if } b^2 - 4ac < 0 \end{cases} \tag{20}$$

When returning the roots, round to two decimal places, *i.e.*, round(x,2). Here are a few examples:

$$q((3,4,2)) = ((-0.67 + 0.47j), (-0.67 - 0.47j)) \tag{21}$$

$$q((1,3,-4)) = (-4.0, 1.0) \tag{22}$$

$$q((1,-2,-4)) = (-1.24, 3.24) \tag{23}$$

Deliverables for Problem 2

- Complete the function according to the equations on line 20.
- Don't forget to round to two decimal places.

Problem 3: Division

In this problem, you'll implement a function that does polynomial division. We will constrain the problem to have the denominator linear. More specifically, we're looking for an answer for this problem:

$$p(x) = a_n x^2 + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0 \tag{24}$$

$$q(x) = b_1 x + b_0 \tag{25}$$

$$f(x) = \frac{p(x)}{q(x)} = c_n x^n + \cdots + g(x) \tag{26}$$

$$g(x) = \begin{cases} 0 & \text{if no remainder} \\ \frac{c_0}{q(x)} & \text{remainder} \end{cases} \tag{27}$$

The function $f(x)$ is built from a series of steps. In this discussion $\leftarrow$ is used for assignment.

1. Let $x = -\frac{b_0}{b_1}$ and $y = a_n$ and answer be the initial list ans $= [y]$
2. Then (as a bounded loop) for $i = n-1, n-2, \ldots, 0$
   (a) $y \leftarrow xy + a_i$
   (b) ans.append(y)

For example,

$$p(x) = 2x^3 - 5x^2 + 3x + 7 \tag{28}$$

$$q(x) = x - 2 \tag{29}$$

$$\frac{p(x)}{q(x)} = 2x^3 - x^2 + x + \frac{9}{x-2} \tag{30}$$

Let's see the steps.

1. $x = -\frac{b_0}{b_1} = -\frac{-2}{1} = 2$
2. $y = a_n = 2$ (from $2x^3$) and ans $= [2]$
3. $y \leftarrow xy + a_{n-1}$ (from $-5x^2$) which is $y \leftarrow 2(2) - 5 = -1$
4. ans $= [2, -1]$
5. $y \leftarrow xy + a_{n-2}$ (from $3x$) which is $y \leftarrow 2(-1) + 3 = 1$
6. ans $= [2, -1, 1]$
7. $y \leftarrow xy + a_{n-3}$ (from 7) which is $y \leftarrow 2(1) + 7 = 9$
8. ans $= [2, -1, 1, 9]$

As you can see from Eq. 30 we have the coefficients. The following code shows how the input is given to the function. Input is a pair of tuples, where the first tuple represents coefficients of p(x), the second tuple contains the coefficients of q(x). The function should return the coefficients of f(x) as shown below:

```
data = [((2,-5,3,7),((1/2),-1)), ((3,0,5,-1),(1,1)), ((4,-8,-1,5), (2,-1))]

for d in data:
    print(sd(*d))
```

has output:

```
[2, -1.0, 1.0, 9.0]
[3, -3.0, 8.0, -9.0]
[4, -6.0, -4.0, 3.0]
```

which represents, respectively:

$$\frac{2x^3 - 5x^2 + 3x + 7}{x - 2} = 2x^3 - x^2 + x + \frac{9}{x - 2} \tag{31}$$

$$\frac{3x^3 + 5x - 1}{x + 1} = 3x^3 - 3x^2 + 8x - \frac{9}{x + 1} \tag{32}$$

$$\frac{4x^3 - 8x^2 - x + 5}{2x - 1} = 4x^3 - 6x^2 - 4x + \frac{3}{2x - 1} \tag{33}$$

$$\tag{34}$$

Deliverables for Problem 3

- Complete the sd function.

Problem 4: Completing the Square

When the roots of a quadratic are real, we can transform the quadratic $ax^2 + bx + c = 0$ into a simpler form and find the roots more easily:

$$ax^2 + bx + c = (x + m)^2 + n$$

To find the roots then:

$$\begin{aligned}
(x+m)^2 + n &= 0 \\
(x+m)^2 &= -n \\
x + m &= \pm\sqrt{-n} \\
x &= -m \pm \sqrt{-n}
\end{aligned}$$

Through some simple algebra we find that

$$\begin{aligned}
m &= \frac{b}{2a} \\
n &= c - \frac{b^2}{4a}
\end{aligned}$$

We can then call our transform function that takes a, b, c and returns m, n as seen below:

```python
def c_s(coefficients):
    pass

def q_(coefficients):
    m,n = c_s(coefficients)
    pass
```

Deliverables for Problem 4

- Complete the functions.
- Round the output from c_s() and q_() function to 2 decimal places.
- The q_ function must call the c_s function.

Problem 5: Toward Statistical Analysis

In this problem, you'll write fundamental statistical functions. We assume a list of numbers $lst = [x_0, x_1, \ldots, x_n]$.

$$
\begin{aligned}
mean(lst) &= (x_0 + x_1 + \cdots + x_n)/\text{len}(lst) && (35)\\
\mu &= mean(lst) && (36)\\
var(lst) &= \frac{1}{\text{len}(lst)}((x_0 - \mu)^2 + (x_1 - \mu)^2 + \cdots + (x_n - \mu)^2) && (37)\\
std(lst) &= \sqrt{var(lst)} && (38)
\end{aligned}
$$

For example, $lst = [1, 3, 3, 2, 9, 10]$, rounding to two places

$$
\begin{aligned}
mean(lst) &= 4.67 && (39)\\
var(lst) &= 12.22 && (40)\\
std(lst) &= 3.5 && (41)
\end{aligned}
$$

The last function mean_centered takes a list of numbers $lst = [x_0, x_1, \ldots, x_n]$ and returns a new list $[x_0 - \mu, x_1 - \mu, \ldots, x_n - \mu]$. An interesting feature of the mean-centered list is that if you try to calculate its mean, it's zero:

$$
\begin{aligned}
\mu &= (x_0 + x_1 + \cdots + x_n)/n && (42)\\
lst &= [x_0 - \mu, x_1 - \mu, \ldots x_n - \mu] && (43)\\
mean(lst) &= ((x_0 - \mu) + \cdots (x_n - \mu))/n && (44)\\
&= ((x_0 + \ldots + x_n) + n\mu)/n && (45)\\
&= \mu - \mu = 0 && (46)
\end{aligned}
$$

Using the same list we have

$$
mean(mean_centered(lst)) = -0.0 = 0 \quad (47)
$$

Deliverables for Problem 5

- Complete the functions.
- Round-up to 2 decimal places the ouptut of mean, var and std functions.

Problem 6: Market Equilibrium

When modeling market behavior we use two curves to indicate supply and demand. You can also think of supply as quantity and demand as want. When the two curves intersect (the common point where they cross), see Fig. 2, there is a point that satisfies both equations. Review the problem of intersecting two lines. Here we have specifically two quadratic functions. Assume

$$s(x) = -0.025x^2 - 0.5x + 60 \tag{48}$$

$$d(x) = 0.02x^2 + 0.6x + 20 \tag{49}$$

for supply s and demand d. We have a function $equi$ that takes the coefficients of s, d and returns the solution:

$$equi((-0.025, -0.5, 60), (0.02, 0.6, 20)) = (20.0, -44.44) \tag{50}$$

Our solution returns the roots to a quadratic equation. Since s, d models the presence of items, only 20 makes sense. HINT: use your q function from Problem 2.

Deliverables for Problem 6

- Complete the function.
- Use q in Problem 2 to make the solution very quick.

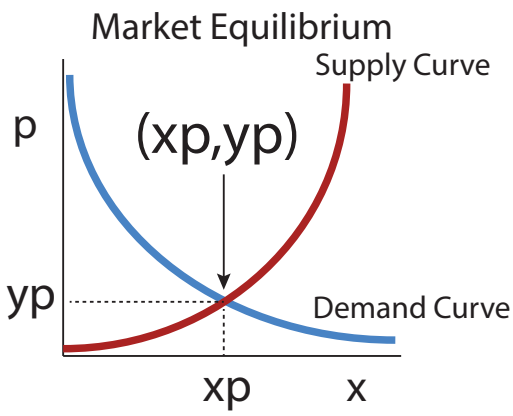


Figure 2: Market equilibrium with supply and demand

Problem 7: Tree Sap

You've been contracted by a law firm to establish ownerhip of property. You've been given a model for tree growth (height in feet) as a function of time (years):

$$h(t) = \frac{120}{1 + 200e^{\alpha t}} \tag{51}$$

$$\alpha = -.2 \tag{52}$$

A dispute involves an owner who claims he planted some trees when his family first acquired the land 20 years ago. You're given permission to measure the height of the two tallest trees measuring $50\,\mathrm{ft}$ and $100\,\mathrm{ft}$. You know the model isn't precise, and so you decide if the time is within half a decade it's probably accurate enough. For example, A tree that is $50\,\mathrm{ft}$ is about 24.81 years old, then if someone claims to have planted it 20 years ago:

$$20 - 5 \leq 24.8 \leq 20 + 5 \tag{53}$$

which is true. On the other hand, if the tree is $100\,\mathrm{ft}$, then

$$20 - 5 \leq 34.54 \leq 20 + 5 \tag{54}$$

which is false. Using the formula for $h(t)$ implement the inverse function, *i.e.*, given a height, what is the time that has elapsed since it was planted. The following code:

```
print(tree_age(50,20))
print(tree_age(100,20))
```

has the output:

```
(24.81, True)
(34.54, False)
```

where the tuple's first component is the life of the tree and the Boolean whether this is within the interval.

Deliverables for Problem 7

- Complete the function.
- Round to two decimal places.

Problem 8: Solving a Small Linear System

A matrix, to remind you, is a rectangular collection of numbers. Matrices play vital roles in AI, finance, physics, *etc.*. A 2×2 square matrix is the smallest useful matrix:

$$\mathbf{x} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \tag{55}$$

A matrix can be effectively captured with Python lists:

```
x = [[a,b],[c,d]]
```

The *determinant* of a matrix is a real number that indicates certain properties. It's calculated by finding the difference between diagonals:

$$\det(\mathbf{x}) = ad - bc \tag{56}$$

Your first task will be to implement a function det() that takes a 2×2 matrix and returns the determinant. For example, this code

```
def determinant(matrix):
    (a,b),(c,d) = matrix
    pass

print(determinant([[1,2],[2,3]]))
```

has output

```
-1
```

Observe that we were able to separate the rows from the matrix! You should experiment and remove the parentheses and observe the behavior. Given two lines

$$ax + by = c \tag{57}$$

$$dx + ey = f \tag{58}$$

we are interested in determining their intersection; that is, so they share a solution $(x*, y*)$. We can use the determinants to find the answer easily. We can consider this a collection of three columns (matrix 1×2):

$$\mathrm{c}_0 = \begin{bmatrix} a \\ d \end{bmatrix} \tag{59}$$

$$\mathrm{c}_1 = \begin{bmatrix} b \\ e \end{bmatrix} \tag{60}$$

$$\mathrm{c}_2 = \begin{bmatrix} c \\ f \end{bmatrix} \tag{61}$$

We can form a new square matrix by combining columns:

$$\mathbf{a} = [\mathsf{c}_2\ \mathsf{c}_1] \tag{62}$$
$$\mathbf{b} = [\mathsf{c}_0\ \mathsf{c}_1] \tag{63}$$
$$\mathbf{d} = [\mathsf{c}_0\ \mathsf{c}_2] \tag{64}$$

Then the solution $(x*, y*)$ is

$$x* = \frac{\mathsf{det}(\mathbf{a})}{\mathsf{det}(\mathbf{b})} \tag{65}$$
$$y* = \frac{\mathsf{det}(\mathbf{d})}{\mathsf{det}(\mathbf{b})} \tag{66}$$

Working through an example:

$$2x + 4y = 11 \tag{67}$$
$$-5x + 3y = 5 \tag{68}$$
$$\mathsf{c}_0 = \begin{bmatrix} 2 \\ -5 \end{bmatrix} \tag{69}$$
$$\mathsf{c}_1 = \begin{bmatrix} 4 \\ 3 \end{bmatrix} \tag{70}$$
$$\mathsf{c}_2 = \begin{bmatrix} 11 \\ 5 \end{bmatrix} \tag{71}$$
$$\mathbf{a} = \begin{bmatrix} 11 & 4 \\ 5 & 3 \end{bmatrix} \tag{72}$$
$$\mathbf{b} = \begin{bmatrix} 2 & 4 \\ -5 & 3 \end{bmatrix} \tag{73}$$
$$\mathbf{d} = \begin{bmatrix} 2 & 11 \\ -5 & 5 \end{bmatrix} \tag{74}$$

Then

$$x* = \frac{11(3) - 4(5)}{2(3) + 5(4)} = \frac{13}{26} = \tfrac{1}{2} \tag{75}$$
$$y* = \frac{2(5) + 5(11)}{26} = \frac{65}{26} = \frac{5}{3} \tag{76}$$

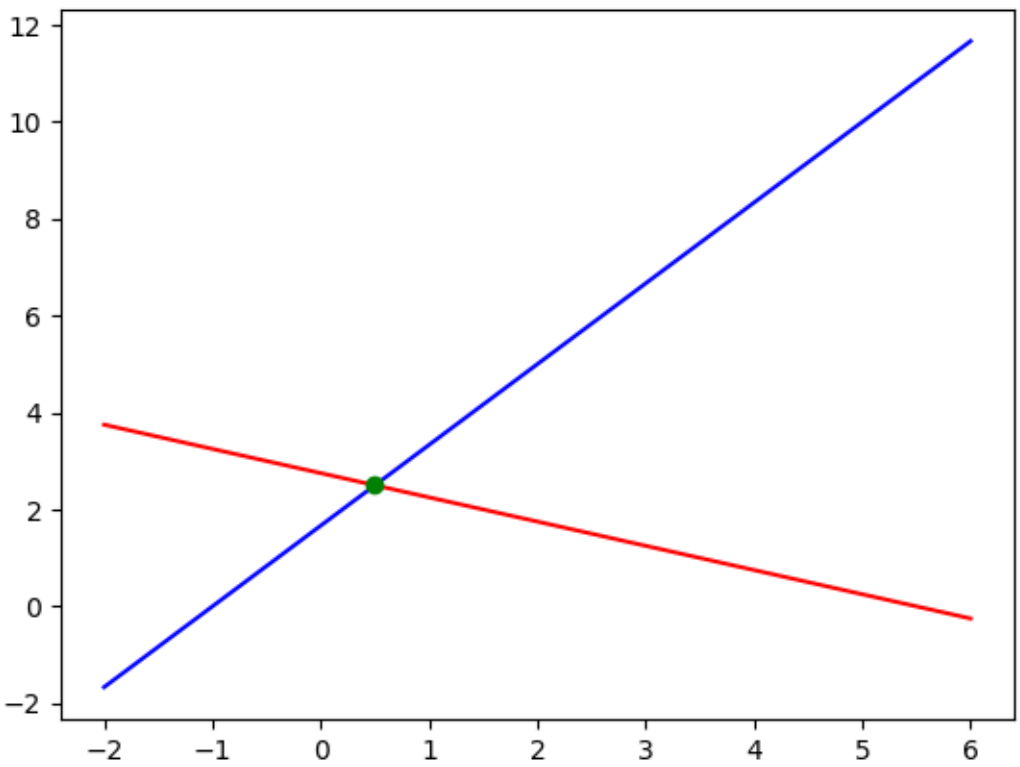
The following code:

```
eq1,eq2 = [[2,4,11],[-5,3,5]]
print(solve(eq1,eq2))
```

produces:

```
(0.5, 2.5)
```

When we plot the image, we see that the intersection (green dot) agrees with our answer.



The code to create the image is very small and uses a module matplotlib (with numpy) that we'll speak to a bit later.

```
import matplotlib.pyplot as plt
import numpy as np

# the first line
def f_1(x):
    return (1/4)*(-2*x + 11)

# the second line
def f_2(x):
    return (1/3)*(5*x + 5)

x_star,y_star = solve(eq1,eq2) #you must implement this function

x = np.linspace(-2,6,100)
plt.plot(x,f_1(x),'r') #plots the first line
plt.plot(x,f_2(x),'b') #plots the second line
plt.plot(x_star,y_star,'go') #plots the green point
plt.show()
```

Deliverables for Problem 8

- Complete the functions determinant and solve
- Round to roots two decimal places
- We will *not* check in Autograder for the case when the determinant is zero

Problem 9: Using Loop to Calculate Sinking Fund Schedule

A **sinking fund** is an financial instrument to discharge a future debt. For example, a manufacturing company expects to replace obsolescent machinary in some number of years. In this problem, you are building your finance company to compute payment schedules for a sinking fund. We use the general formula for annuity:

$$\begin{aligned} S &= R\frac{(1+i)^n - 1}{i} \\ i &= \frac{r}{m} \\ n &= my \end{aligned}$$

where S is the future debt, r the (percentage) interest rate, m the number of payments per year, y the number of years, n is the total number of payments (number of payments in a year multiplied by total number of years) and R is the monthly deposit made. Here is the particular problem. Acme Sundials believes replacing their sun dial mold-press will cost $30,000 in two years. They can currently get 10% interest compounded quarterly (four times a year). You build a Python program that, given the initial data will:

- Determine the monthly deposit (R).
- Determine the total schedule of payments that includes the interest accrued and total amount. This should be stored as a list of lists (see example output below).

Here is the run:

```
def deposit(S,r,i,n):
   pass

def sinking_fund(final_amt,r,m,y):
    pass

for i in sinking_fund(30000,.1,4,2):
    print(i)
```

with output:

```
[[0, 3434.02, 0, 3434.02],
[1, 3434.02, 85.85, 6953.89],
[2, 3434.02, 173.85, 10561.76],
[3, 3434.02, 264.04, 14259.82],
[4, 3434.02, 356.5, 18050.34],
[5, 3434.02, 451.26, 21935.62],
[6, 3434.02, 548.39, 25918.03],
[7, 3434.02, 647.95, 30000.0]]
```

- The function deposit returns the monthly deposit needed for the fund (R).
- Clearly you must solve for R (monthly deposit) first and use this value in sinking_fund() to solve for S.
- Once you have the deposit value, you should find n and i.
- After you have all the values, i..e, i, n, R, you can start calculating your payment schedule.
- Make sure that the payment schedule is stored as a list of lists where each sublist represent one payment as follows: [period, deposit, interest, total fund].

Observe there are eight payments. On the last payment, the fund is $30,000. The first value in the list is the period, the second is the deposit amount, the third is the interest accrued on the **previous** fund, the last value is the curent total fund. Let's construct period 1 from 0. The total fund in period 0 is $3434.02, or the initial deposit value for this particular example.

$$\begin{aligned} i \times fund &= \frac{r}{m} \times 3434.02 \\ &= \frac{.10}{4} \times 3434.02 = 0.025(3434.02) = 85.85 \end{aligned}$$

The total t_1 for that period is the previous total, plus interest, plus deposit:

$$t_1 = 3434.02 + 85.85 + 3434.02 = 6953.89$$

Deliverables for Problem 9

- Complete the functions deposit and sinking_fund.
- We are rounding to two decimal places outout from deposit function, value of interest and total fund.
- You are free to use a single bounded (hint), unbounded, nested loops–whatever you find most comfortable.

Problem 10: Geometric Progression

A geometric progression is described by $ar^0, ar^1, ar^2, \ldots$ for positive integers a, r. The ratio of successive terms is a constant *viz.*, $\frac{ar^1}{ar^0}, \frac{ar^2}{ar^1}, \ldots$. Thus, given a (finite) sequence we can determine whether it is geometric or not. In this problem you'll implement a function is_geometric_sequence(lst) that takes a possibly empty list of numbers. If there are at least three, then you can determine if it is a geometric series. For example,

```
data = [[1,2,4,6],[2,4,8,16],[10,30,90,270,810,2430]]
for d in data:
    print(is_geometric_sequence(d))
```

produces

```
False
True
True
```

Deliverables for Problem 10

- Complete the function.
- As always you cannot use any exotic or functions not discussed in class.

Problem 11: Portfolio Valuation

In this problem, you'll implement a function value(portfolio, market) that takes a person's portfolio of stocks and the current market and determines the value as a percent. A stock is a portion of ownership of a business. Since it's a portion, it is a fraction of what the business is worth in total. A stock as a price per share as a dollar amount and the number of shares (how much of the portion is owned). For example, if you have a stock that you purchased for $2.25/share and you purchased 11 shares then $\$2.25 \times 11 = \$24.75$ in total. A portfolio consists of the stock name, share price that was purchased, and the number of shares. We'll model this a dictionary:

```
portfolios =  {'A':{'stock':{'x':(41.45,45),'y':(22.20,1000)}},
                    'B':{'stock':{'x':(33.45,15),'y':(12.20,400)}}}
```

shows two portfolios, one for 'A' and one for 'B'. 'A' has 45 shares of 'x' valued at $41.45 and 'y' valued at $22.20 with 1000 shares. The total value is: $41.45(45) + $22.20(1000) = $24065.25. The market value for stocks changes. In this example, our market is:

```
market = {'x':43.00, 'y':22.50}
```

This means that 'A's holdings in 'x' is worth now $\$43.00 \times 45 = \$1935.0$ instead of $\$41.45 \times 45 = 1865.25$. The value of a portfolio is the ratio of $\frac{\text{Market}-\text{Portfolio}}{\text{Portfolio}}$. If we were solely looking at the 'x' stock, then

$$\frac{\text{Market} - \text{Portfolio}}{\text{Portfolio}} = \frac{43.00(45) - 41.45(45)}{41.45(45)} \tag{77}$$

$$\approx 3.7\% \tag{78}$$

which means the value today is 3.7% more. Implement the function value(portfolio, market) that determines the change in percentage of worth. For example,

```
portfolios =  {'A':{'stock':{'x':(41.45, 45),'y':(22.20, 1000)}},'B':{'stock':{'x':(33.45,15),'y':(12.20,400)}}}

market = {'x':43.00, 'y':22.50}

for name, portfolio in portfolios.items():
    print(f"{name} {value(portfolio,market)}")
```

produces

```
A 2.0
B 79.0
```

that gives 2% for A and 79% for B.

Deliverables for Problem 11

- Complete the function.
- The function returns a percent.
- Round to two decimal places.

Problem 12: Smoothing Data

When data is temporal (ordered by time), we sometimes want to “smooth" it by giving a simple average. The simplest is taking two consecutive values and finding their average. Given a (possibly empty) list of numbers lst $= [x_0, x_1, \ldots, x_n]$ the function smooth(lst) returns:

- The string “AveError: list empty" if the list is empty
- lst if list only contains one number
- a new list of numbers $[y_0, y_1, \ldots, y_{n-1}]$ such that $y_i = \frac{x_i + x_{i+1}}{2}$

The following code

```
data = [[], [1],[1,2],[1,2,2,3],[0,2,4,6,8]]
for d in data:
    print(smooth(d))
```

produces

```
AveError: list empty
[1]
[1.5]
[1.5, 2.0, 2.5]
[1.0, 3.0, 5.0, 7.0]
```

Deliverables for Problem 12

- Complete the function.
- Round to two decimal places.

Problem 13: Space, the Final Frontier

The weight of an astronaut whose altitude is d kilometers above the ground weighs approximately:

$$A_w(E_w, d) \quad = \quad E_w \left(\frac{6400}{6400 + d} \right)^2 \tag{79}$$

where E_w is the weight on earth. Implement a function that can give the minimal altitude an astronaut must attain given both the weight on earth and weight at distance distance (d) from earth. For example, a 165 pound (earth weight) astronaut will weight 5 pounds if $d \geq 30,366$ kilometers from the earth's surface. The following code:

```
print(orbit(5,165))
```

produces

```
30366
```

Deliverables for Problem 13

- Complete the function.
- Use the math ceiling function for the solution.

Problem 14: Come Fly with Me (Sinatra)

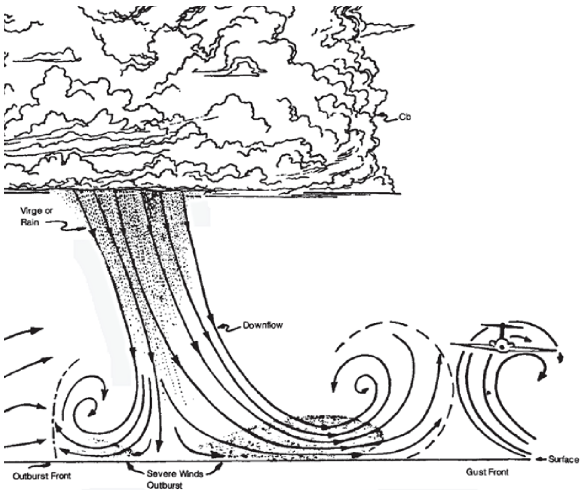


Figure 3: Image taken from FAA

You've been hired by the FAA to calculate wind shear contants. Wind shear is a meterological phenomenon where the velocity of wind v_0 at height h_0 and the velocity of wind v_1 at height h_1 differ significantly. Wind shear affects airplanes as they take-off and land. The formula for verticle wind sheer is:

$$\frac{v_0}{v_1} = (\frac{h_0}{h_1})^P \tag{80}$$

Implement a function P(v0,h0,v1,h1) that takes wind velocities and heights and returns the constant value of P. The following function:

```
v0,h0 = 25,200
v1,h1 = 6,35
print(P_ws(v0,h0,v1,h1))
```

produces

```
0.819
```

Deliverables for Problem 14

- Complete the function
- Round the final answer to three decimal places

Programming partners

reychaud@iu.edu, ashahi@iu.edu
aadkhan@iu.edu, leorusyn@iu.edu
nsfried@iu.edu, mvsimi@iu.edu
jdemian@iu.edu, frvisco@iu.edu
sdharmmi@iu.edu, snlnu@iu.edu
amgandhi@iu.edu, pratmodi@iu.edu
askaul@iu.edu, ptapiaro@iu.edu
logellis@iu.edu, ap59@iu.edu
jabbarke@iu.edu, andmuchi@iu.edu
cdinman@iu.edu, kammich@iu.edu
akumares@iu.edu, nvoneper@iu.edu
chenkay@iu.edu, bsai@iu.edu
ykassahu@iu.edu, fshamrin@iu.edu
ecanant@iu.edu, lawlin@iu.edu
anudivek@iu.edu, animerch@iu.edu
daveanu@iu.edu, nsenthi@iu.edu
hedavid@iu.edu, traepena@iu.edu
jakedeck@iu.edu, bfmark@iu.edu
sebcont@iu.edu, zstopher@iu.edu
burkhel@iu.edu, adpulipa@iu.edu
arkali@iu.edu, bpovinel@iu.edu
kjj6@iu.edu, nicmill@iu.edu
njustin@iu.edu, ceprenti@iu.edu
nkushnir@iu.edu, mackeyr@iu.edu
adevill@iu.edu, abbalokh@iu.edu
rcerauli@iu.edu, mmaleck@iu.edu
careyjo@iu.edu, amtsmith@iu.edu
lineri@iu.edu, avaweish@iu.edu
blberger@iu.edu, benserna@iu.edu
lasdodd@iu.edu, wagnebr@iu.edu
ml82@iu.edu, khwillar@iu.edu
mgrass@iu.edu, arstover@iu.edu
allentp@iu.edu, ggriorda@iu.edu
mbadwe@iu.edu, crschuit@iu.edu
samjimen@iu.edu, bwmoore@iu.edu
auconner@iu.edu, cmoverfe@iu.edu
nbkotur@iu.edu, brshelby@iu.edu
nabatja@iu.edu, jusshah@iu.edu
arkhal@iu.edu, camlueck@iu.edu

presharr@iu.edu, nsmurthy@iu.edu
okorkmaz@iu.edu, cmm23@iu.edu
benitezb@iu.edu, tiffyou@iu.edu
lbadredd@iu.edu, jocpoiri@iu.edu
jadebail@iu.edu, tryerrag@iu.edu
kespejo@iu.edu, nmetlapa@iu.edu
bcabral@iu.edu, javitto@iu.edu
changali@iu.edu, erugova@iu.edu
aydclink@iu.edu, arangi@iu.edu
zcarmich@iu.edu, smunikun@iu.edu
anchinna@iu.edu, rahkpate@iu.edu
balian@iu.edu, ishetty@iu.edu
sambereb@iu.edu, katepate@iu.edu
eberens@iu.edu, trimedio@iu.edu
cbremner@iu.edu, vupatel@iu.edu
cedonica@iu.edu, surehma@iu.edu
wbrogan@iu.edu, etmillho@iu.edu
abelani@iu.edu, teswank@iu.edu
forlin@iu.edu, rtripat@iu.edu
gmhowell@iu.edu, syeshala@iu.edu
josajack@iu.edu, stewti@iu.edu
jkchaura@iu.edu, jianlou@iu.edu
agunde@iu.edu, martichc@iu.edu
jaxkatz@iu.edu, saravula@iu.edu
sgudapur@iu.edu, camyee@iu.edu
ryandaun@iu.edu, ymarrakc@iu.edu
shbivens@iu.edu, jamread@iu.edu
zlangley@iu.edu, twirges@iu.edu
zaycarey@iu.edu, iopfings@iu.edu
camehugh@iu.edu, jomalek@iu.edu
chodgman@iu.edu, mgstack@iu.edu
kimyouh@iu.edu, msantell@iu.edu
chamblia@iu.edu, auspiege@iu.edu
craftdc@iu.edu, ksreejit@iu.edu
achitred@iu.edu, armnguy@iu.edu
marbelli@iu.edu, tmukund@iu.edu
pkurzawa@iu.edu, wmccarte@iu.edu
edjgarci@iu.edu, mitcchar@iu.edu
allennam@iu.edu, amoza@iu.edu
wchorich@iu.edu, msolomie@iu.edu
kylrguth@iu.edu, breramer@iu.edu

berrynm@iu.edu, nrimawi@iu.edu
ankotel@iu.edu, shshar@iu.edu
reafrank@iu.edu, archshar@iu.edu
emiellis@iu.edu, haoztian@iu.edu
cfahy@iu.edu, rimaeda@iu.edu
renahan@iu.edu, joprayit@iu.edu
adeglur@iu.edu, alexpan@iu.edu
cgreywal@iu.edu, shmulla@iu.edu
nkhai@iu.edu, weinstab@iu.edu
andhodge@iu.edu, ntorpoco@iu.edu
jamlian@iu.edu, chsabir@iu.edu
jhbruce@iu.edu, jmatczak@iu.edu
pafrost@iu.edu, joebrahm@iu.edu
ecabebe@iu.edu, olipfing@iu.edu
cjduffie@iu.edu, njvinson@iu.edu
boding@iu.edu, menonvi@iu.edu
blevidut@iu.edu, smremmer@iu.edu
eynkansa@iu.edu, ctsvetan@iu.edu
ocrowder@iu.edu, sparab@iu.edu
sjbarry@iu.edu, wangjinz@iu.edu
tkalale@iu.edu, wardjohn@iu.edu
harrisdg@iu.edu, shubtiwa@iu.edu
belleli@iu.edu, gmwerlin@iu.edu
conkeen@iu.edu, corrosar@iu.edu
beckab@iu.edu, johnsanc@iu.edu
rkeel@iu.edu, rsreedh@iu.edu
jvgottum@iu.edu, xsoto@iu.edu
lrborows@iu.edu, jlopezmo@iu.edu
rachguo@iu.edu, lomwean@iu.edu
shaagore@iu.edu, amatariy@iu.edu
langgrav@iu.edu, aquintas@iu.edu
kayfaris@iu.edu, leoett@iu.edu
madimond@iu.edu, manushah@iu.edu
markdeng@iu.edu, mxrobins@iu.edu
jailjohn@iu.edu, gravipat@iu.edu
spebrook@iu.edu, reddysk@iu.edu
cc96@iu.edu, peterabl@iu.edu
tkonkim@iu.edu, peterant@iu.edu
harmonnc@iu.edu, masoroki@iu.edu
ejbusch@iu.edu, gstogdil@iu.edu
smhamrah@iu.edu, colbstap@iu.edu